

Progettazione Concettuale, Logica, DDL e Normalizzazione

Esercitazione di Basi di Dati

May 25, 2026

Section 1

Introduzione all'esercitazione

Cosa faremo oggi:

- **Esempio 1 — Biblioteca:** ER da zero, traduzione logica, DDL, errori classici
- **Esempio 2 — Ordini e Prodotti:** normalizzazione esplicita $1NF \rightarrow 2NF \rightarrow 3NF$, anomalie

Prima di iniziare: creare un database di lavoro in DBeaver chiamato `esercizio_db`.

Creare il database

```
sudo -u postgres psql
```

```
-- Eseguire sul server come superuser (postgres)
```

```
CREATE DATABASE esercitazione_db;
```

```
-- Assegna i permessi (privilegi) all'utente sul database
```

```
ALTER DATABASE esercitazione_db OWNER TO dbman;
```

Connettersi e creare uno schema per ogni esempio

```
-- Connessione a esercitazione_db
```

```
psql -h localhost -U dbman -d esercitazione_db
```

```
-- Creazione di 2 schemi:
```

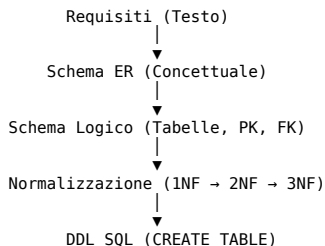
```
CREATE SCHEMA biblioteca;
```

```
CREATE SCHEMA ordini;
```

```
-- Impostare il search_path per l'esempio corrente
```

```
SET search_path TO biblioteca;
```

In DBeaver: tasto destro sul database → SQL Editor → New SQL Script



Regola: *Non iniziare mai a scrivere SQL prima di aver capito cosa rappresentano i dati.*

Section 2

Esempio 1: Biblioteca

Una biblioteca gestisce un catalogo di libri. Ogni libro ha un titolo, un anno di pubblicazione e un genere. Un libro può essere scritto da uno o più autori; di ogni autore interessa nome, cognome e nazionalità. La biblioteca ha dei soci (nome, cognome, email, data di iscrizione). I soci possono prendere libri in prestito: si registra la data del prestito e la data di restituzione (che può essere nulla se il libro non è ancora stato restituito). Un socio può avere più prestiti nel tempo.

Prima cosa da fare: sottolineare i sostantivi (potenziali entità) e i verbi (potenziali relazioni).

Regola pratica: un'entità è tale, se ha *almeno un attributo proprio* oltre alla chiave e *partecipa a relazioni*.

Dai sostantivi del testo:

Candidato	Entità?	Motivazione
Libro	✓ Sì	Ha attributi propri, è centrale
Autore	✓ Sì	Ha attributi propri, è indipendente
Socio	✓ Sì	Ha attributi propri
Prestito	✓ Sì	Ha attributi propri (date), non è solo relazione
Genere	? Forse	Dipende: semplice attributo o entità separata?
Biblioteca	× No	È il sistema stesso, non un'entità da modellare

Scelta su “Genere”: attributo o entità?

Opzione A — Attributo semplice:

```
LIBRO(isbn, titolo, anno, genere)
-- genere = 'Romanzo', 'Saggistica', ...
```

Opzione B — Entità separata:

```
LIBRO(isbn, titolo, anno, id_genere) —FK→ GENERE(id_genere, nome_genere)
```

Quando scegliere B?

- Se il genere ha *attributi propri* (es. descrizione, codice Dewey)
- Se serve *referential integrity* stretta sui valori ammessi
- Se la lista è *gestita centralmente* (aggiornamenti propagati)

Per questo esercizio: useremo l’Opzione B — è più istruttiva e mostra le FK.

⚠ **Errore comune:** modellare come entità tutto ciò che sembra un “tipo” (colore, stato, categoria). Chiedetevi sempre: ha senso da solo?

Come gestire le diverse “copie” di un libro?

Soluzione 1: Contatori in LIBRO

Aggiungere gli attributi `copie_totali` e `copie_disponibili` nella tabella LIBRO.

- **Pro:**

- Struttura semplice con una sola tabella.
- Lettura immediata della disponibilità senza operazioni di JOIN.

- **Contro:**

- Impossibile tracciare lo stato della singola copia (es. se è danneggiata).
- Impossibile associare i prestiti storici a uno specifico oggetto fisico.
- Rischio di disallineamento dei contatori in caso di errori software durante i prestiti.

Soluzione 2: Entità COPIA

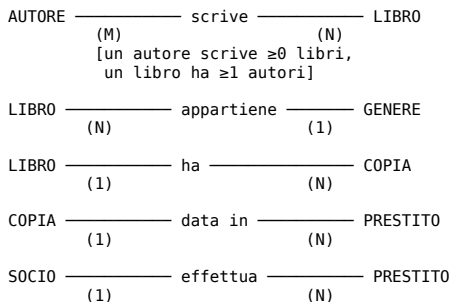
Creare una tabella separata COPIA legata a LIBRO tramite una relazione 1:N.

- **Pro:**

- Tracciabilità granulare di ogni volume (posizione sullo scaffale, stato di usura).
- Gestione dei prestiti solida basata sull'identificativo della singola copia.
- Calcolo dinamico e sicuro della disponibilità tramite funzioni di aggregazione (COUNT).

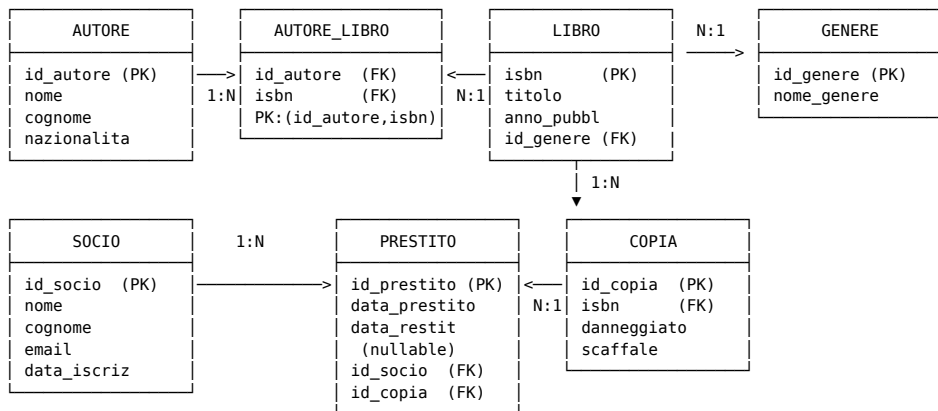
- **Contro:**

- Schema più complesso che richiede la gestione di una tabella aggiuntiva.
- Query di lettura più onerose a causa della necessità di utilizzare le JOIN.



*La relazione M:N **AUTORE-LIBRO**. Come la gestiamo?*

Schema logico completo



Tre regole fondamentali della traduzione ER → Relazionale:

① **Ogni entità** → una tabella con la sua chiave primaria

② **Relazione 1:N** → chiave esterna nella tabella lato N

`LIBRO(isbn, ..., id_genere) -- id_genere FK → GENERE`

③ **Relazione M:N** → nuova tabella con le due FK come chiave composta

`AUTORE_LIBRO(id_autore FK, isbn FK)
-- PK = (id_autore, isbn)`

⚠ **Perché la tabella ponte:** Se non usassimo la tabella ponte per le M:N dovremmo mettere gli autori come stringa separata da virgole nel campo del libro. Vediamo perché è sbagliato →

Come NON fare

```
-- \texttimes SBAGLIATO: attributo multivalore
CREATE TABLE libro_sbagliato (
  isbn      VARCHAR(13) PRIMARY KEY,
  titolo    TEXT,
  autori    TEXT    -- es: P. Atzeni, S. Ceri, P. Fraternali, S. Paraboschi, R. Torlone
);
```

Problemi:

- Come cerco tutti i libri di Eco? → LIKE '%Eco%' → fragile, lento, sbagliato
- Come aggiorno il nome di un autore? → update manuale su ogni riga
- Viola la **Prima Forma Normale (1NF)**: ogni cella deve contenere un valore atomico

1. Prima Forma Normale (1NF) — Atomicità

- **Regola:** Ogni colonna contiene un solo valore. No liste, array o gruppi ripetuti. Generi e autori multipli non sono stipati come testo in LIBRO, ma isolati nelle tabelle GENERE e AUTORE_LIBRO.

2. Seconda Forma Normale (2NF) — Dipendenza totale

- **Regola:** Gli attributi dipendono dall'intera chiave primaria, non da una sua parte (no dipendenze parziali).
Rispettata in automatico nelle tabelle a chiave singola. La tabella a chiave composta AUTORE_LIBRO PK: (id_autore, isbn) non ha attributi descrittivi.

3. Terza Forma Normale (3NF) — No transitività

- **Regola:** Gli attributi dipendono solo dalla PK, senza intermediari non-chiave.

DDL — Schema biblioteca (1/3): Tabelle di base

```
SET search_path TO biblioteca;
```

```
-- Genere letterario
```

```
CREATE TABLE genere (  
    id_genere    SERIAL    PRIMARY KEY,  
    nome_genere  VARCHAR(50) NOT NULL UNIQUE  
);
```

```
-- Autore
```

```
CREATE TABLE autore (  
    id_autore    SERIAL    PRIMARY KEY,  
    nome         VARCHAR(50) NOT NULL,  
    cognome      VARCHAR(50) NOT NULL,  
    nazionalita  VARCHAR(50),  
    -- vincolo: nome+cognome non duplicati  
    UNIQUE (nome, cognome)  
);
```

```
-- Socio della biblioteca (utente)
```

```
CREATE TABLE socio (  
    id_socio     SERIAL    PRIMARY KEY,  
    nome         VARCHAR(50) NOT NULL,  
    cognome      VARCHAR(50) NOT NULL,  
    email        VARCHAR(100) NOT NULL UNIQUE,  
    data_iscriz  DATE      NOT NULL DEFAULT CURRENT_DATE  
);
```

DDL — Schema biblioteca (2/3): Libro e tabella ponte

```
-- Libro
CREATE TABLE libro (
    isbn          VARCHAR(13)  PRIMARY KEY,
    titolo        TEXT         NOT NULL,
    anno Pubbl   SMALLINT      CHECK (anno Pubbl > 0 AND anno Pubbl <= 2100),
    id_genere     INT          REFERENCES genere(id_genere)
                                ON DELETE SET NULL
);

-- Tabella ponte M:N Autore-Libro
CREATE TABLE autore_libro (
    id_autore     INT          NOT NULL REFERENCES autore(id_autore)
                                ON DELETE RESTRICT,
    isbn          VARCHAR(13)  NOT NULL REFERENCES libro(isbn)
                                ON DELETE CASCADE,
    PRIMARY KEY (id_autore, isbn)
);

-- Ogni riga è un oggetto fisico con un "codice a barre" (id_copia)
CREATE TABLE copia (
    id_copia      SERIAL PRIMARY KEY,
    isbn          VARCHAR(13) REFERENCES libro(isbn),
    danneggiato   BOOLEAN DEFAULT FALSE, -- true se il libro è in restauro o perso
    scaffale      VARCHAR(20)  -- Es: 'Sala A - Scaffale 3'
);
```

Notare: ON DELETE CASCADE sul libro — se elimino il libro, via le righe ponte. ON DELETE RESTRICT sull'autore — non posso eliminare un autore che ha libri.

DDL — Schema biblioteca (3/3): Prestito e vincolo unicità

```
-- Prestito
CREATE TABLE prestito (
    id_prestito    SERIAL      PRIMARY KEY,
    id_socio       INT         NOT NULL REFERENCES socio(id_socio)
                                ON DELETE RESTRICT,
    id_copia       INT         NOT NULL REFERENCES copia(id_copia)
                                ON DELETE RESTRICT,
    data_prestito  DATE        NOT NULL DEFAULT CURRENT_DATE,
    data_restit    DATE,       -- NULL = libro non ancora restituito

    -- Vincolo: data restituzione >= data prestito
    CONSTRAINT chk_date CHECK (
        data_restit IS NULL OR data_restit >= data_prestito
    )
);
```

Popoliamo il DB

-- 1. Genere

```
INSERT INTO genere (nome_genere) VALUES
('Fantascienza'),
('Romanzo Storico'),
('Giallo'),
('Saggistica'),
('Fantasy');
```

-- 2. Autore

```
INSERT INTO autore (nome, cognome, nazionalita) VALUES
('Isaac', 'Asimov', 'Statunitense'),
('Umberto', 'Eco', 'Italiana'),
('Agatha', 'Christie', 'Britannica'),
('J.R.R.', 'Tolkien', 'Britannica'),
('Alessandro', 'Manzoni', 'Italiana');
```

-- 3. Socio

```
INSERT INTO socio (nome, cognome, email, data_iscriz) VALUES
('Mario', 'Rossi', 'mario.rossi@email.it', '2023-01-15'),
('Giulia', 'Bianchi', 'g.bianchi@email.it', '2023-02-20'),
('Luca', 'Verdi', 'luca.verdi@email.com', '2023-03-10'),
('Anna', 'Neri', 'anna.neri@email.it', '2023-05-05'),
('Paolo', 'Gialli', 'p.gialli@email.com', '2023-06-12');
```

-- 4. Libro

-- L'id genere fa riferimento agli ID generati sopra (1=Fantascienza, 2=Storico, ecc.)

```
INSERT INTO libro (isbn, titolo, anno Pubbl, id_genere) VALUES
('9788804668237', 'Fondazione', 1951, 1),
('9788845292613', 'Il nome della rosa', 1980, 3),
('9788804704027', 'Dieci piccoli indiani', 1939, 3),
('9788845292614', 'Il Signore degli Anelli', 1954, 5),
('9788806228323', 'I Promessi Sposi', 1827, 2);
```

Popoliamo il DB (2)

```
-- 5. Autore_Libro (Tabella Ponte)
-- Associa gli autori ai rispettivi libri
INSERT INTO autore_libro (id_autore, isbn) VALUES
(1, '9788804668237'), -- Asimov -> Fondazione
(2, '9788845292613'), -- Eco -> Il nome della rosa
(3, '9788804704027'), -- Christie -> Dieci piccoli indiani
(4, '9788845292614'), -- Tolkien -> Il Signore degli Anelli
(5, '9788806228323'); -- Manzoni -> I Promessi Sposi

-- 6. Copia
-- Alcuni libri hanno più copie
INSERT INTO copia (isbn, danneggiato, scaffale) VALUES
('9788804668237', FALSE, 'Sala A - Scaffale 1'),
('9788804668237', FALSE, 'Sala A - Scaffale 1'), -- Seconda copia di Fondazione
('9788845292613', TRUE, 'Sala B - Scaffale 2'), -- Libro danneggiato
('9788804704027', FALSE, 'Sala C - Scaffale 1'),
('9788845292614', FALSE, 'Sala A - Scaffale 3');

-- 7. Prestito
-- id_copia fa riferimento alle 5 copie create sopra.
-- data_restit a NULL significa che il libro è attualmente in prestito.
INSERT INTO prestito (id_socio, id_copia, data_prestito, data_restit) VALUES
(1, 1, '2023-09-01', '2023-09-15'), -- Restituito
(2, 4, '2023-09-10', '2023-09-20'), -- Restituito
(3, 2, '2023-10-01', NULL), -- In corso
(4, 5, '2023-10-05', NULL), -- In corso
(1, 3, '2023-10-10', '2023-10-12'); -- Restituito
```

Come ottengo le copie totali di un libro? Come faccio a tenere il conto delle copie disponibili?

```
SELECT l.titolo, COUNT(*) AS copie_totali
FROM libro l
JOIN copia c ON l.isbn = c.isbn
WHERE l.titolo = 'Il nome della rosa'
GROUP BY l.titolo;
```

```
SELECT l.titolo, COUNT(*) AS copie_disponibili
FROM libro l
LEFT JOIN copia c ON l.isbn = c.isbn
  AND c.danneggiato = FALSE
  AND c.id_copia NOT IN (
    -- Esclude gli ID delle copie che sono attualmente in prestito
    SELECT p.id_copia
    FROM prestito p
    WHERE p.data_restit IS NULL
  )
WHERE LOWER(l.titolo) = 'il nome della rosa'
GROUP BY l.titolo;
```


Section 3

Esempio 2: Ordini e Prodotti

*Un'azienda vuole gestire i propri **ordini**. Ogni ordine è fatto da un **cliente** e contiene uno o più **prodotti** con la rispettiva quantità ordinata. Di ogni prodotto si conosce il nome, la categoria e il **fornitore** (uno solo per prodotto), con nome e città del fornitore. Il cliente ha nome, indirizzo, CAP, provincia e città. Ogni ordine ha una data.*

Scenario: il database è già esistente, fatto male, e dobbiamo *normalizzarlo*. Partiamo da una tabella unica (come succederebbe esportando da un foglio Excel).

```
id_ordine,data_ordine,id_cliente,nome_cliente,indirizzo_cli,citta_cliente,id_prodotti,nomi_prodotti
1,2024-01-10,101,Mario Rossi,Via Roma 1,Milano,"501, 502","Notebook Dell, Mouse Logitech","2, 3","899
2,2024-01-15,101,Mario Rossi,Via Roma 1,Milano,"501","Notebook Dell","1","899.00"
3,2024-01-20,102,Anna Bianchi,Corso Italia 5,Roma,"503","Tastiera Apple","1","129.00"
```

Normalizzazione: Recap (1NF - BCNF)

Forma Normale	Regola	Violazione	Soluzione generale
1NF	Ogni cella è atomica; no liste, no gruppi ripetuti	telefoni = '333-111, 02-999'	Più righe o tabella separata
2NF	1NF + ogni attributo dipende dall'intera chiave	PK (A, B) → C dipende solo da A	Scindere in due tabelle
3NF	2NF + no dipendenze transitive tra non-chiave	[Non-Chiave 1] → [Non-Chiave 2] (Una colonna fa da "dizionario" a un'altra)	Creare una tabella anagrafica dove [Non-Chiave 1] diventa PK
BCNF	3NF + ogni determinante è una superchiave	[Non-Chiave] → [Pezzo di PK] (Le dipendenze "tornano indietro" verso la chiave)	Isolare il determinante in una nuova tabella come PK

La tabella “piatta” di partenza

```
SET search_path TO ordini;
```

```
CREATE TABLE ordine_raw (  
    id_ordine          INT PRIMARY KEY,  
    data_ordine        DATE,  
    id_cliente         INT,  
    nome_cliente       VARCHAR(100),  
    indirizzo_cli      VARCHAR(200),  
    citta_cliente      VARCHAR(50),  
    cap_cli            CHAR(5),  
    provincia_cli      CHAR(2),  
    id_prodotti         TEXT, -- Es: '501, 502'  
    nomi_prodotti       TEXT, -- Es: 'Notebook Dell, Mouse Logitech'  
    categorie_prodotti  TEXT, -- Es: 'Elettronica, Accessori'  
    id_fornitori        TEXT, -- Es: '301, 302'  
    nomi_fornitori      TEXT, -- Es: 'Tech Srl, Accessori Spa'  
    citta_fornitori     TEXT, -- Es: 'Torino, Roma'  
    quantita           TEXT,  -- Es: '2, 3'  
    prezzi_unitari      TEXT  -- Es: '899.00, 25.00'  
);
```

Dati nella tabella piatta

```
INSERT INTO ordine_raw VALUES
```

```
(1, '2024-01-10', 101, 'Mario Rossi', 'Via Roma 1', 'Milano', '20100', 'MI',  
'501, 502', 'Notebook Dell, Mouse Logitech', 'Elettronica, Accessori',  
'301, 302', 'Tech Srl, Accessori Spa', 'Torino, Roma',  
'2, 3', '899.00, 25.00'),
```

```
(2, '2024-01-15', 101, 'Mario Rossi', 'Via Roma 1', 'Milano', '20100', 'MI',  
'501', 'Notebook Dell', 'Elettronica',  
'301', 'Tech Srl', 'Torino',  
'1', '899.00'),
```

```
(3, '2024-01-20', 102, 'Anna Bianchi', 'Corso Italia 5', 'Roma', '00100', 'RM',  
'503', 'Tastiera Apple', 'Accessori',  
'302', 'Accessori Spa', 'Roma',  
'1', '129.00');
```

Proviamo a rispondere:

- Come calcolo il totale dell'ordine 1?
 - Splitting delle stringhe in qtà e prezzi_unit, cast ad INT/DECIMAL, solo dopo il calcolo.
- Come cerco chi ha comprato il 'Mouse Logitech'?
 - Devo usare LIKE '%Mouse Logitech%': lento (full table scan) e prone a errori ("Tappetino per Mouse Logitech"?).

Anomalia di aggiornamento:

```
-- Mario Rossi si trasferisce a Torino. Update dell'ultimo ordine
UPDATE ordine_raw
SET citta_cliente = 'Torino', indirizzo_cli = 'Via Po 10'
WHERE id_ordine = 2; -- L'impiegato agisce sull'ordine corrente
-- Aggiorna una riga... 2 entry di Mario Rossi con città/indirizzo diversi
```

*Se aggiorniamo solo 1 riga su 2, abbiamo **dati inconsistenti**: lo stesso cliente ha due indirizzi diversi nel database.*

Anomalia di inserimento:

```
-- Vogliamo registrare un nuovo fornitore che non ha ancora ordini.
INSERT INTO ordine_raw (id_ordine, id_fornitori, nomi_fornitori)
VALUES (NULL, '301', 'Tech Srl');
-- ERRORE: id_ordine non può essere NULL!
-- NON POSSIAMO: ogni riga richiede un ordine.
```

Anomalia di cancellazione:

```
-- Se cancelliamo l'ordine 3 (l'unico di Anna Bianchi),
-- perdiamo anche tutte le informazioni su Anna Bianchi.
DELETE FROM ordine_raw WHERE id_ordine = 3;
-- Anna Bianchi non esiste più nel database!
```

Una tabella è in 1NF se:

- ❶ ✓ Ogni cella contiene un valore **atomico** (non liste, non array)
- ❷ ✓ Ogni riga è **unica** (c'è una chiave)
- ❸ ✓ Non ci sono **gruppi ripetuti** (colonne tipo prodotto1, prodotto2, ...)

La nostra ordine_raw è in 1NF?

- Valori atomici: `x(id_prodotti = '501, 502', nomi_prodotti = 'Notebook Dell, Mouse Logitech' — sono liste!)`
- Riga unica: ✓ (`id_ordine` è PRIMARY KEY)
- Gruppi ripetuti: `x(tutti i prodotti compressi in campi TEXT)`

*La tabella **NON** è in 1NF.*

*Le anomalie derivano da **dipendenze funzionali** non rispettate.*

Dipendenze Funzionali

id_cliente → nome_cliente, indirizzo_cli, citta_cliente
id_prodotto → nome_prodotto, categoria_prod, id_fornitore
id_fornitore → nome_fornitore, citta_forn
id_ordine → data_ordine, id_cliente
(id_ordine, id_prodotto) → quantita, prezzo_unitario

Analizziamo la dipendenza: id_prodotto → nome_prodotto.

Nella tabella ordine_raw, la chiave che comanda la riga è id_ordine. Ma il nome_prodotto non dipende dall'ordine, dipende dall' id_prodotto! Cosa succede se tre clienti diversi in tre ordini diversi comprano il prodotto 501?

id_ordine	...	id_prodotto	nomi_prodotti
---	---	---	---
1	...	501	Notebook Dell
2	...	501	Notebook Dell
3	...	501	Notebook Dell

In questa tabella, la dipendenza id_prodotto → nome_prodotto esiste concettualmente, ma non è strutturalmente imposta dal DDL: nulla vieta di inserire Dell PC invece di Notebook Dell con lo stesso id_prodotto.

id_ordine	...	id_prodotto	nomi_prodotti
---	---	---	---
1	...	501	Notebook Dell
2	...	501	Notebook Dell
3	...	501	Dell PC

Passo 1: Portare in Prima Forma Normale (1NF)

- ❶ **Atomicità:** ogni cella deve contenere un solo valore singolo.
- ❷ **Unicità:** ogni riga deve essere identificabile da una chiave.

Per applicare la 1NF dobbiamo “esplodere” le liste, creando una riga distinta per ogni combinazione Ordine-Prodotto.

```
CREATE TABLE ordine_1nf (  
    id_ordine      INT,  
    data_ordine    DATE,  
    id_cliente     INT,  
    nome_cliente   VARCHAR(100),  
    cap_cli        CHAR(5),  
    indirizzo_cli  VARCHAR(200),  
    citta_cliente  VARCHAR(50),  
    provincia_cli  CHAR(2),  
    id_prodotto    INT,  
    nome_prodotto  VARCHAR(100),  
    categoria_prod VARCHAR(50),  
    id_fornitore   INT,  
    nome_fornitore VARCHAR(100),  
    citta_forn     VARCHAR(50),  
    quantita       INT,  
    prezzo_unitario DECIMAL(10,2),  
  
    PRIMARY KEY (id_ordine, id_prodotto) -- chiave COMPOSTA  
);
```

```
INSERT INTO ordine_1nf VALUES
```

```
-- Ordine 1, cliente Rossi, 2 prodotti → 2 righe distinte
```

```
(1, '2024-01-10', 101, 'Mario Rossi', '20100', 'Via Roma 1', 'Milano', 'MI',  
501, 'Notebook Dell', 'Elettronica', 301, 'Tech Srl', 'Torino', 2, 899.00),
```

```
(1, '2024-01-10', 101, 'Mario Rossi', '20100', 'Via Roma 1', 'Milano', 'MI',  
502, 'Mouse Logitech', 'Accessori', 302, 'Accessori Spa', 'Roma', 3, 25.00),
```

```
-- Ordine 2, stesso cliente Rossi
```

```
(2, '2024-01-15', 101, 'Mario Rossi', '20100', 'Via Roma 1', 'Milano', 'MI',  
501, 'Notebook Dell', 'Elettronica', 301, 'Tech Srl', 'Torino', 1, 899.00),
```

```
-- Ordine 3, cliente Bianchi
```

```
(3, '2024-01-20', 102, 'Anna Bianchi', '00100', 'Corso Italia 5', 'Roma', 'RM',  
503, 'Tastiera Apple', 'Accessori', 302, 'Accessori Spa', 'Roma', 1, 129.00);
```

Quante righe ha il cliente Rossi? Quante volte appare il suo indirizzo? Cosa succede se cambia città?

Definizione 2NF

Una tabella è in 2NF se è in 1NF e ogni attributo non-chiave dipende dall'**intera** chiave primaria, non da una sua parte soltanto.

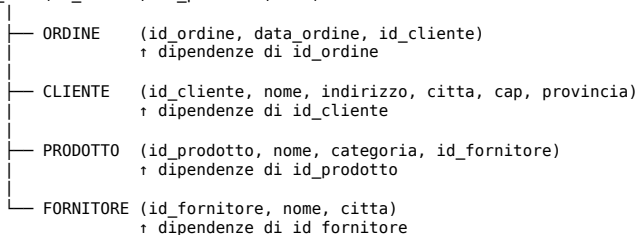
La chiave di ordine_1nf è (id_ordine, id_prodotto).

Attributo	Dipende da...	Problema?
quantita	(id_ordine, id_prodotto) ✓	No
prezzo_unitario	(id_ordine, id_prodotto) ✓	No
data_ordine	id_ordine solo ×	Sì: dipendenza parziale
nome_cliente	id_cliente (via id_ordine) ×	Sì
nome_prodotto	id_prodotto solo ×	Sì: dipendenza parziale
nome_fornitore	id_fornitore solo ×	Sì

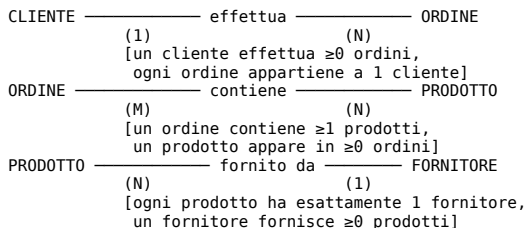
Soluzione 2NF: separare ogni gruppo di attributi nella propria tabella.

Dal testo del problema emergono 4 entità distinte:

ordine_Inf (id_ordine, id_prodotto, ...)



*Queste entità sono “naturalì”: emergono direttamente dal testo, ognuna con i propri attributi. Ma **ORDINE** e **PRODOTTO** come si collegano?*



*La relazione $M : N$ **ORDINE-PRODOTTO**: come la gestiamo?*

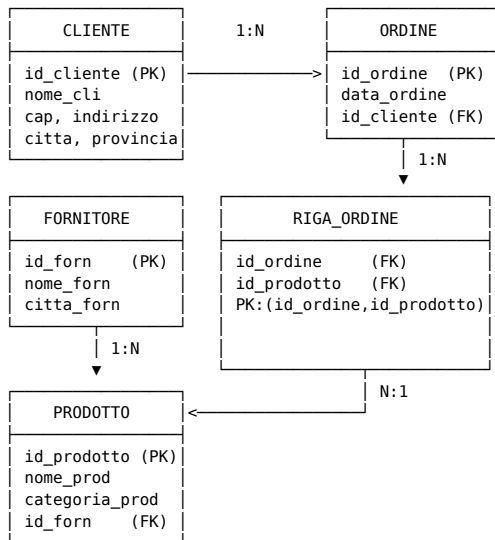
Per risolvere la relazione $M : N$, introduciamo una tabella intermedia che trasforma una relazione difficile in due relazioni $1 : N$ pulite:

ORDINE $\xrightarrow{(1:N)}$ RIGA_ORDINE $\xrightarrow{(N:1)}$ PRODOTTO

- Un ordine appare in **molte** righe di RIGA_ORDINE
- Un prodotto appare in **molte** righe di RIGA_ORDINE
- Ogni riga di RIGA_ORDINE lega **esattamente un** ordine a **esattamente un** prodotto

*La chiave primaria della tabella ponte è **composta**: (*id_ordine*, *id_prodotto*)*

Schema Logico (fino ad ora)



Dove Mettiamo gli Attributi “Orfani”?

Dalla tabella piatta originale restano due attributi: `quantita` e `prezzo_unitario`. In quale tabella li inseriamo?

Proviamo in PRODOTTO:

*Se nel prodotto “Notebook Dell” scriviamo `quantita` = 2, stiamo dicendo che chiunque lo compri è obbligato a prenderne due. **Non ha senso**: la quantità non è una proprietà intrinseca dell’oggetto.*

Proviamo in ORDINE:

*Se nell’ordine #1 scriviamo `quantita` = 2 e `prezzo` = 899, a quale prodotto si riferiscono? Al Notebook o al Mouse? Non possiamo saperlo. **L’ordine da solo** descrive solo quando e da chi.*

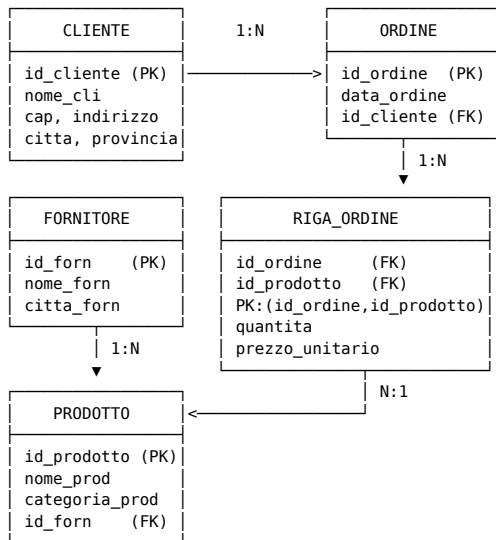
La quantità e il prezzo non appartengono all'ordine *in sé*, e non appartengono al prodotto *in sé*.

Appartengono al **legame** tra quell'*ordine specifico* e quel *prodotto specifico*.

Il legame risiede esattamente nella tabella ponte che abbiamo appena creato per risolvere la $M : N$.

Aggiungendo quantità e prezzo_unitario alla tabella ponte, questa smette di essere vuota.

La Struttura in 2NF



RIGA_ORDINE è il cuore: ogni riga risponde a “**quante** unità di **quale** prodotto in **quale** ordine, a **quale** prezzo?”

Chiave di RIGA_ORDINE: (id_ordine, id_prodotto)

Attributo	Dipende da...	Problema?
quantita	(id_ordine, id_prodotto) ✓	No
prezzo_unitario	(id_ordine, id_prodotto) ✓	No

Nelle altre tabelle la chiave è semplice (non composta): una dipendenza parziale non è nemmeno possibile.

DDL — Schema ordini (1/3): Entità indipendenti

```
SET search_path TO ordini;
```

```
CREATE TABLE fornitore (  
    id_fornitore SERIAL PRIMARY KEY,  
    nome_forn VARCHAR(100) NOT NULL,  
    città_forn VARCHAR(50)  
);
```

```
CREATE TABLE prodotto (  
    id_prodotto SERIAL PRIMARY KEY,  
    nome_prod VARCHAR(100) NOT NULL,  
    categoria_prod VARCHAR(50),  
    id_fornitore INT NOT NULL  
        REFERENCES fornitore(id_fornitore)  
        ON DELETE RESTRICT,  
    prezzo_listino DECIMAL(10,2) -- prezzo "di catalogo"  
);
```

```
-- Versione 2NF di cliente (città e provincia dirette).
```

```
-- Vedremo nella 3NF perché questo crea ridondanza.
```

```
CREATE TABLE cliente (  
    id_cliente SERIAL PRIMARY KEY,  
    nome_cli VARCHAR(100) NOT NULL,  
    indirizzo VARCHAR(200),  
    città VARCHAR(50),  
    cap CHAR(5),  
    provincia CHAR(2)  
);
```

DDL — Schema ordini (2/3): Ordine e riga ordine

```
CREATE TABLE ordine (  
    id_ordine    SERIAL    PRIMARY KEY,  
    data_ordine  DATE      NOT NULL DEFAULT CURRENT_DATE,  
    id_cliente   INT       NOT NULL  
                  REFERENCES cliente(id_cliente)  
                  ON DELETE RESTRICT  
);  
  
-- Tabella ponte con attributi propri.  
-- prezzo_unitario è il prezzo storico al momento dell'ordine  
-- (il prezzo di listino nel catalogo potrebbe cambiare in futuro).  
CREATE TABLE riga_ordine (  
    id_ordine    INT       NOT NULL  
                  REFERENCES ordine(id_ordine)  
                  ON DELETE CASCADE,  
    id_prodotto  INT       NOT NULL  
                  REFERENCES prodotto(id_prodotto)  
                  ON DELETE RESTRICT,  
    quantita     INT       NOT NULL,  
    prezzo_unitario DECIMAL(10,2) NOT NULL,  
  
    PRIMARY KEY (id_ordine, id_prodotto),  
  
    CONSTRAINT chk_qta    CHECK (quantita > 0),  
    CONSTRAINT chk_prezzo CHECK (prezzo_unitario >= 0)  
);
```

Terza Forma Normale (3NF)

Definizione: una tabella è in 3NF se è in 2NF e non esistono **dipendenze transitive**: nessun attributo non-chiave dipende da un altro attributo non-chiave.

Controlliamo la tabella cliente:

cliente(id_cliente, nome_cli, indirizzo, citta, cap, provincia)

Dipendenza	Tipo
id_cliente → citta	Transitiva: id_cliente → cap → citta
id_cliente → provincia	Transitiva: id_cliente → cap → provincia

Assumiamo che il CAP (o il macro-CAP) determini univocamente il comune e la provincia (non propriamente vero per le grandi città).
citta e provincia non dipendono direttamente da id_cliente, ma dal CAP — un attributo non-chiave.

Soluzione: estrarre una tabella cap_comune(cap, citta, provincia).

Violazione 3NF — Perché è un problema nel nostro caso

Situazione attuale (non in 3NF): città e provincia sono in cliente ed hanno una dipendenza da CAP (non-chiave).

Supponiamo di avere questi dati:

id_cliente	nome_cli	cap	città	provincia
1	Mario Rossi	20100	Milano	MI
2	Anna Bianchi	20100	Milano	MI
3	Luca Verdi	20100	Milano	MI

Anomalia di aggiornamento — il comune di Milano cambia nome amministrativo:

```
UPDATE cliente SET città = 'Milano Città Metropolitana'
WHERE cap = '20100';
-- Quante righe devo aggiornare? Tutte quelle con quel CAP.
-- Se dimentico una riga → stesso CAP, due città diverse nel DB.
```

Anomalia di inserimento — voglio registrare che il CAP 74100 è sempre Taranto:

```
-- Non posso: non ho un cliente con quel CAP da inserire.
-- L'informazione "74100 → Taranto" esiste solo se esiste un cliente.
```

3NF — Esempio pratico con CAP (opzione 1)

```
-- Versione NON in 3NF (dipendenza transitiva: cap → città, provincia)
/*CREATE TABLE cliente (
    id_cliente SERIAL PRIMARY KEY,
    nome_cli VARCHAR(100) NOT NULL,
    indirizzo VARCHAR(200),
    cap CHAR(5),
    città VARCHAR(50), -- ← dipende da cap, non da id_cliente!
    provincia CHAR(2) -- ← idem
);*/
```

-- Soluzione 3NF: separare il CAP in tabella propria

```
CREATE TABLE cap_comune (
    cap CHAR(5) PRIMARY KEY,
    città VARCHAR(50) NOT NULL,
    provincia CHAR(2) NOT NULL
);
```

```
/* Versione Cliente 3NF
CREATE TABLE cliente (
    id_cliente SERIAL PRIMARY KEY,
    nome_cli VARCHAR(100) NOT NULL,
    indirizzo VARCHAR(200),
    cap CHAR(5) REFERENCES cap_comune(cap)
);*/
```

ALTER TABLE cliente

-- 1. Rimuovi le colonne che ora dipendono da cap_comune

DROP COLUMN città,

DROP COLUMN provincia,

-- 2. Aggiungi il vincolo di chiave esterna sulla colonna cap esistente

ADD CONSTRAINT fk_cliente_cap

FOREIGN KEY (cap) REFERENCES cap_comune(cap);

3NF — Esempio pratico con CAP (opzione 2)

-- Sostituisce la versione 2NF di cliente

```
CREATE TABLE cap_comune (  
  cap      CHAR(5)      PRIMARY KEY,  
  citta    VARCHAR(50)  NOT NULL,  
  provincia CHAR(2)     NOT NULL  
);
```

```
DROP TABLE IF EXISTS cliente CASCADE; -- rimuove la versione 2NF
```

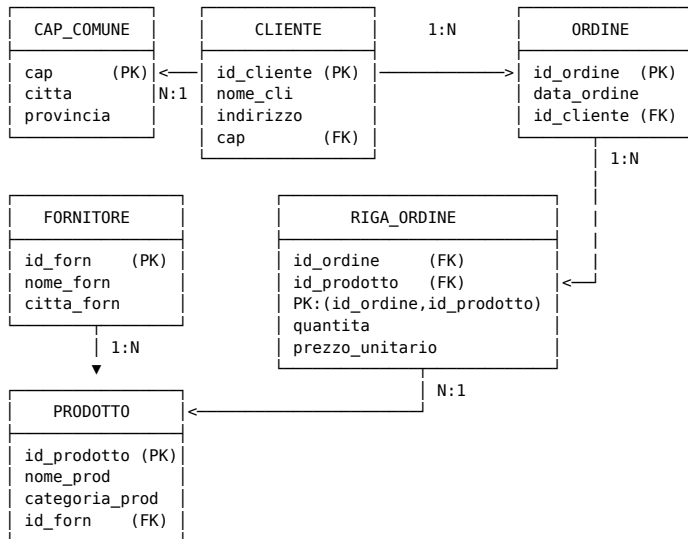
```
CREATE TABLE cliente (  
  id_cliente SERIAL      PRIMARY KEY,  
  nome_cli   VARCHAR(100) NOT NULL,  
  indirizzo  VARCHAR(200),  
  cap        CHAR(5)     REFERENCES cap_comune(cap)  
  -- citta e provincia si ricavano con JOIN su cap_comune  
);
```

-- Ricrea il collegamento tra ordine e cliente (eliminato dal DROP CASCADE)

```
ALTER TABLE ordine  
ADD CONSTRAINT fk_cliente_id  
  FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
```

Le tabelle fornitore, prodotto, ordine e riga_ordine non subiscono modifiche: erano già in 3NF.

La Struttura Finale in 3NF



Lo Schema è anche in BCNF?

Definizione: una tabella è in BCNF se è in 3NF e per ogni dipendenza funzionale del tipo $X \rightarrow Y$ (non banale), la parte sinistra X deve essere una superchiave: nessun attributo può dipendere da qualcosa che non sia una chiave candidata della tabella.

Se ogni città ha uno e un solo (macro)-CAP. (Es: Milano ha il macro-CAP 20100, ignorando le zone postali).

In questo scenario, si crea una **corrispondenza biunivoca (1:1)** tra il CAP e la Città.

① **cap $\rightarrow$ città, provincia**

- *Analisi BCNF:* Il determinante (cap) è la Chiave Primaria. Essendo la PK, è una **Superchiave**. (Regola rispettata)

② **città, provincia $\rightarrow$ cap** (La dipendenza duale)

- *Analisi BCNF:* Poiché una città ha un solo CAP, non ci saranno mai righe duplicate per la stessa città. La combinazione (città, provincia) diventa univoca, ovvero una **Chiave Candidata**. Essendo una chiave candidata, è una **Superchiave**. (Regola rispettata)

Come dimostrato, entrambi i determinanti (cap e (città, provincia)) sono **chiavi candidate**, quindi superchiavi. Poiché ogni determinante è una superchiave, la tabella cap_comune soddisfa **per definizione** la BCNF.

Popolare il DB normalizzato (1/2)

-- Rispettare l'ordine: prima le tabelle senza FK (foglie dello schema)

INSERT INTO fornitore (nome_forn, citta_forn) **VALUES**

('Tech Srl', 'Torino'), -- id = 1

('Accessori Spa', 'Roma'); -- id = 2

INSERT INTO cap_comune (cap, citta, provincia) **VALUES**

('20100', 'Milano', 'MI'),

('00100', 'Roma', 'RM');

INSERT INTO cliente (nome_cli, indirizzo, cap) **VALUES**

('Mario Rossi', 'Via Roma 1', '20100'), -- id = 1

('Anna Bianchi', 'Corso Italia 5', '00100'); -- id = 2

INSERT INTO prodotto (nome_prod, categoria_prod, id_fornitore, prezzo_listino) **VALUES**

('Notebook Dell', 'Elettronica', 1, 899.00), -- id = 1

('Mouse Logitech', 'Accessori', 2, 25.00), -- id = 2

('Tastiera Apple', 'Accessori', 2, 129.00); -- id = 3

```
INSERT INTO ordine (data_ordine, id_cliente) VALUES  
( '2024-01-10', 1),  
( '2024-01-15', 1),  
( '2024-01-20', 2);
```

```
-- I dati prima compressi nelle liste TEXT  
-- ora sono righe distinte con tipi corretti
```

```
INSERT INTO riga_ordine (id_ordine, id_prodotto, quantita, prezzo_unitario) VALUES  
(1, 1, 2, 899.00), (1, 2, 3, 25.00),  
(2, 1, 1, 899.00), (3, 3, 1, 129.00);
```

“Chi ha comprato il Mouse Logitech?”

Prima (versione raw):

```
-- Cerca dentro stringhe: lento, fragile, case-sensitive
SELECT nome_cliente
FROM ordine_raw
WHERE nomi_prodotto LIKE '%Mouse Logitech%';
-- E se il prodotto si chiamasse 'mouse logitech'?
-- E se il nome fosse 'Super Mouse Logitech Pro'?
```

Adesso (schema normalizzato):

```
SELECT DISTINCT c.nome_cli
FROM cliente c
JOIN ordine o ON c.id_cliente = o.id_cliente
JOIN riga_ordine ro ON o.id_ordine = ro.id_ordine
JOIN prodotto p ON ro.id_prodotto = p.id_prodotto
WHERE p.nome_prod = 'Mouse Logitech';
```

Ricerca esatta su un indice, non una scansione di stringhe.

Prima era complesso: Calcolare il totale

“Qual è l'importo totale dell'ordine 1?”

Prima (versione raw):

```
WITH elementi_esplosi AS (  
    SELECT  
        id_ordine,  
        -- unnest() esplode l'array in righe verticali.  
        -- Ciascun unnest genererà una riga per ogni elemento dell'array.  
        trim(unnest(string_to_array(quantita, ','))::INT AS qta,  
        trim(unnest(string_to_array(prezzi_unitari, ','))::DECIMAL(10,2) AS prezzo  
    FROM ordine_raw  
    WHERE id_ordine = 1 -- Ci concentriamo solo sull'ordine 1  
)  
SELECT  
    id_ordine,  
    SUM(qta * prezzo) AS totale_ordine  
FROM elementi_esplosi  
GROUP BY id_ordine;
```

Adesso (schema normalizzato):

```
SELECT  
    o.id_ordine,  
    c.nome_cli,  
    SUM(ro.quantita * ro.prezzo_unitario) AS totale_ordine  
FROM ordine o  
JOIN cliente c ON o.id_cliente = c.id_cliente  
JOIN riga_ordine ro ON o.id_ordine = ro.id_ordine  
WHERE o.id_ordine = 1  
GROUP BY o.id_ordine, c.nome_cli;
```

Il processo

- 1 Leggere i requisiti
- 2 Schema ER (entità, relazioni, cardinalità)
- 3 Schema logico (tabelle, PK, FK)
- 4 Normalizzazione (1NF → 3NF)
- 5 DDL SQL

Traduzione ER → Relazionale

- **Entità** → tabella con PK
- **1:N** → FK nel lato N
- **M:N** → tabella ponte con PK composta

Forme normali

- 1NF** Valori atomici, no liste
- 2NF** Dipendenza dall'intera PK
- 3NF** No transitività tra non-chiave
- BCNF** Ogni determinante è superchiave

Errori da evitare

- Campi TEXT con liste di valori
- Iniziare dal DDL senza ER
- Modellare come entità ciò che è solo attributo